UNIVERSIDAD NACIONAL DE LA MATANZA

Departamento de Ingeniería e Investigaciones Tecnológicas

Trabajo práctico · Programación concurrente y métricas de IA

SISTEMA CONCURRENTE DE RESERVA DE ENTRADAS PARA CINE

CINEMA RESERVATION SYSTEM

Diseño productor–consumidor, sincronización de recursos y evaluación del desarrollo asistido por inteligencia artificial

Integrantes

43262563 - Abaca, Ivan Fabrizio

44396900 - Barcella, Joaquin Antu

43572601 - Coronati, Augusto Martin

43895910 – Gonzalez, Luca Sebastian

43458509 - Licarzi, Florencia Berenice

43508768 – Zabalgoitia Agustin

Asignatura: Programación Concurrente (3658) – Com 01-3900

Universidad Nacional de La Matanza · San Justo, Buenos Aires, Argentina

RESUMEN

Este trabajo presenta el desarrollo y la evaluación de Cinema Reservation System, una aplicación de escritorio para la reserva concurrente de entradas de cine elaborada con asistencia de inteligencia artificial. El sistema, implementado en Python y Tkinter, aplica el patrón productor–consumidor mediante hilos, colas sincronizadas y un bloqueo individual por asiento para evitar ventas duplicadas. Se describe la especificación técnica suministrada a la herramienta de IA, la organización modular del código y las pruebas efectuadas en dos escenarios de carga. En verificaciones independientes de 600 solicitudes, tanto la carga distribuida como la alta contención procesaron la totalidad de los pedidos sin duplicar reservas. Asimismo, la prueba específica de 20 solicitudes sobre un único asiento obtuvo una reserva exitosa y 19 rechazos en tres repeticiones. Las métricas registradas por el equipo muestran consumos medios de CPU de 5,5 % y 6,0 % y un pico de memoria de 35 MB en ambos escenarios. Finalmente, se examinan las rúbricas, las limitaciones del prototipo y las mejoras pendientes.

Palabras clave: *programación concurrente, productor–consumidor, exclusión mutua, reserva de asientos, inteligencia artificial*

Documento preparado sobre la base del código fuente entregado, el prompt técnico, la planilla de métricas y las dos rúbricas. Los resultados de verificación complementaria están diferenciados de las mediciones originalmente informadas.

I. CONTEXTO

El presente proyecto se desarrolla en el ámbito académico de la Universidad Nacional de La Matanza y pertenece al área de la ingeniería de software, específicamente a la programación concurrente y a la evaluación de aplicaciones generadas con asistencia de inteligencia artificial. Su objeto es construir un sistema didáctico de reservas capaz de atender solicitudes simultáneas sin comprometer la integridad de los asientos disponibles. No se identifican en la documentación suministrada entidades financiadoras ni clientes externos; el catálogo y las transacciones son simulados [1], [2].

La dificultad central surge cuando dos usuarios intentan reservar el mismo asiento de una función en un intervalo de tiempo cercano. Si la verificación de disponibilidad y la actualización del estado se realizan sin coordinación, ambas solicitudes podrían registrar la misma venta. La solución adoptada separa la producción de solicitudes de su procesamiento, utiliza una cola segura para hilos y restringe el acceso exclusivo al asiento concreto que se modifica [1], [2].

II. INTRODUCCIÓN

La concurrencia permite que varias unidades de ejecución progresen durante un mismo intervalo de tiempo, incluso cuando comparten recursos. En un sistema de reservas, los productores representan usuarios que solicitan entradas y los consumidores representan trabajadores que evalúan dichas peticiones. El patrón productor–consumidor desacopla ambas tareas mediante una cola, evitando que la interfaz de usuario modifique por sí misma el inventario [6], [7].

El riesgo técnico más significativo es la condición de carrera: dos workers podrían consultar un asiento disponible antes de que cualquiera registre su reserva. Para eliminar esa posibilidad, la verificación y el cambio de estado deben efectuarse como una única sección crítica protegida por un mecanismo de exclusión mutua. El uso de un bloqueo individual por asiento evita serializar operaciones independientes en otros lugares de la sala [1], [2], [6].

Además de verificar la corrección concurrente, este estudio registra indicadores de uso de CPU y memoria e interpreta dos rúbricas institucionales: funcionalidad de la aplicación y mecanismos de concurrencia [3]–[5]. Se considera también la utilización de inteligencia artificial para producir el código a partir de una especificación detallada, sin asumir que la generación automática elimina la necesidad de revisión y pruebas.

III. MÉTODOS

A. Estrategia de desarrollo asistido por IA

La planilla de proyecto identifica la herramienta Antigravity, el modelo declarado Gemini 3.8, el lenguaje Python y la metodología de “prompt técnico”. Consigna tres refinamientos y 24.770 tokens registrados mediante cálculo matemático. No se adjuntó una exportación de consumo de la plataforma ni el texto individual de cada refinamiento; en consecuencia, los 24.770 tokens se tratan como una estimación declarada, no como un conteo auditado automáticamente [3].

El prompt entregado delimita el desarrollo a Python 3.12 o superior, programación orientada a objetos, threading.Thread para productores y consumidores, queue.Queue para solicitudes, threading.Lock por asiento y Tkinter para la interfaz. Excluye bases de datos, autenticación, pagos, APIs externas, multiprocessing y asyncio. También exige pruebas repetibles, dos escenarios de simulación y un cierre ordenado [1]. El alcance se mantuvo deliberadamente local y en memoria.

B. Arquitectura y componentes

El código fuente se organiza en siete módulos, con responsabilidades diferenciadas. El archivo principal inicia el catálogo, crea cuatro consumidores y luego abre la ventana de la aplicación. Los modelos incluyen solicitudes con identificador, usuario, película, función, asiento y marca temporal, además de estados y resultados [2].

TABLA I. COMPONENTES DE LA SOLUCIÓN Y RESPONSABILIDADES

Módulo	Responsabilidad principal
main.py	Inicialización del catálogo, los workers y la interfaz.
models.py	Tipos de estado; estructuras de solicitud y resultado.
cinema.py	Catálogo de películas, funciones y asientos, con Lock por asiento.
reservation_system.py	Cola de solicitudes, consumidores, contadores y cola de resultados.
simulation.py	Productores concurrentes, pruebas distribuida/conflicto y sincronización de la carga.
ui.py	Interfaz Tkinter, selección, mapa de asientos, historial y contadores.
medir_metricas.py	Medición de CPU y memoria con psutil, automática o por PID externo.

El catálogo precargado consta de Matrix, Inception e Interstellar. Cada película dispone de dos funciones y cada función de 30 asientos distribuidos en cinco filas y seis columnas. En total existen 180 asientos lógicos, independientes por combinación de película y horario. Así, la reserva de A1 en Matrix a las 20:00 no afecta al A1 de otra función [2].

C. Procesamiento y sincronización

Cada productor genera objetos ReservationRequest y los incorpora a una única cola compartida. Cuatro workers persistentes, iniciados mediante threading.Thread, extraen solicitudes de esa cola y consultan el asiento solicitado. Si existe, el worker adquiere su Lock particular mediante el bloque with, verifica el estado AVAILABLE y lo modifica a RESERVED cuando corresponde; las peticiones posteriores sobre ese mismo asiento se registran como rechazadas. Los contadores compartidos se actualizan mediante un bloqueo independiente, metrics_lock [2], [6], [7].

TABLA II. SECUENCIA DE PROCESAMIENTO CONCURRENTE

1. Origen	2. Encolado	3. Ejecución	4. Sección crítica	5. Presentación
Usuario / productores	request_queue (Queue)	Workers Thread (×4)	Lock del asiento → reserva o rechazo	result_queue → Tkinter

Los workers publican los resultados en una segunda cola. El hilo de la interfaz consulta periódicamente esta cola mediante after(), registra eventos en el historial y actualiza los indicadores de pendientes, generadas, procesadas, exitosas y rechazadas. La separación entre cola de trabajo y cola de resultados favorece la capacidad de respuesta de la ventana [2].

La protección se aplica a un único asiento por operación, sin adquirir simultáneamente varios bloqueos de asientos; esta decisión reduce tanto la contención innecesaria como el riesgo de interbloqueos. Se utiliza queue.task_done() y queue.join() para esperar el procesamiento íntegro de las cargas de simulación. El cierre de consumidores se implementa con elementos centinela None y join() sobre los workers [2].

D. Diseño experimental y criterios de medición

El prompt define dos cargas: (i) distribuida, en la que las solicitudes se reparten aleatoriamente entre películas, funciones y asientos, y (ii) alta contención, en la que todos los productores intentan reservar Matrix, función 20:00, asiento A1. Contempla además una prueba específica de 20 solicitudes concurrentes contra ese único asiento, cuyo criterio correcto es una reserva exitosa y 19 rechazos [1], [2].

Como comprobación complementaria del código recibido, se ejecutaron dos simulaciones independientes, cada una con un catálogo nuevo, 30 productores, 20 solicitudes por productor y cuatro workers, además de tres repeticiones de la prueba de conflicto. Estas pruebas se realizaron sobre la lógica de Python sin abrir la interfaz gráfica. Por separado, se utilizaron los datos históricos de CPU y memoria ya consignados por el equipo en la planilla entregada [2], [3].

El script medir_metricas.py emplea psutil y muestrea periódicamente CPU y memoria RSS del proceso aproximadamente cada 0,05 segundos. A efectos de interpretación, la convención consignada es 100 % de CPU

equivalente a un núcleo lógico. El script admite tanto ejecución automatizada como supervisión por PID; la planilla no discrimina cuál de esas modalidades se utilizó para producir cada valor anotado [2], [3], [8].

IV. RESULTADOS Y OBJETIVOS

A. Resultados funcionales

El análisis estático del código confirma la presencia de selección de película y función, campo de usuario, mapa visual de asientos con colores para disponible y reservado, reserva manual por clic, dos botones de simulación y prueba específica de conflicto. También se implementan historial de decisiones de los workers y contadores de estado. La aplicación está diseñada para ejecutarse localmente con python main.py; en esta revisión no se realizó una validación visual interactiva de Tkinter [2].

B. Pruebas de procesamiento concurrente

TABLA III. VERIFICACIONES COMPLEMENTARIAS SOBRE EL CÓDIGO ENTREGADO

Prueba	Solicitudes	Procesadas	Exitosas	Rechazadas
Carga distribuida (30 × 20)	600	600	174	426
Alta contención (30 × 20)	600	600	1	599
Conflicto, repetición 1	20	20	1	19
Conflicto, repetición 2	20	20	1	19
Conflicto, repetición 3	20	20	1	19

Nota. Las dos simulaciones de 600 solicitudes se ejecutaron por separado con un catálogo inicialmente disponible. La cantidad de éxitos en carga distribuida depende de las elecciones aleatorias. La prueba de conflicto reinicia la función antes de cada repetición. Estas verificaciones son complementarias y no equivalen a un ensayo exhaustivo de todas las situaciones de la interfaz.

En ambas simulaciones de 600 solicitudes se cumplió la igualdad solicitudes generadas = solicitudes procesadas. Tampoco se detectaron asientos con más de una reserva exitosa para la misma película y función. En alta contención, el único asiento elegido registró una aceptación y 599 rechazos. La verificación específica de 20 peticiones reprodujo el resultado esperado en tres ejecuciones. Los resultados respaldan la protección de la sección crítica en los escenarios ensayados, aunque no constituyen una demostración formal de ausencia de condiciones de carrera bajo cualquier intercalación [2].

C. Métricas de CPU y memoria registradas

TABLA IV. CONSUMO DE RECURSOS INFORMADO EN LA PLANILLA

Indicador	Carga distribuida	Alta contención
CPU promedio	5,5 %	6,0 %
CPU máximo observado	74,4 %	66,5 %
Memoria informada (pico RSS)	35 MB	35 MB
CPU lógicas del equipo	16	16
Memoria física total del equipo	32.660 MB	32.660 MB

Los valores consignados corresponden al método psutil y al criterio de que 100 % de CPU representa un núcleo lógico. El promedio de CPU difiere en 0,5 puntos porcentuales entre los dos escenarios, mientras que el pico registrado es 7,9 puntos mayor en la carga distribuida. La memoria máxima informada coincide en 35 MB. Estas diferencias describen las corridas registradas, sin que puedan inferirse tendencias estadísticamente concluyentes: la planilla no incluye series temporales, repeticiones documentadas, desviaciones estándar ni intervalos de confianza [3].

La medición de CPU y memoria debe interpretarse junto con el diseño de la carga: los productores y workers incorporan pausas breves para simular comportamiento de usuarios y latencia. Por ello, el consumo observado no representa necesariamente el techo de rendimiento del algoritmo ni habilita una comparación directa con una

implementación sin asistencia de IA. Tampoco se documentó en la planilla el tiempo real de desarrollo o una referencia temporal de programación manual [2], [3].

D. Evaluación conforme a las rúbricas

TABLA V. RÚBRICA DE FUNCIONALIDAD: CALIFICACIONES REGISTRADAS

Criterio	Nivel	Interpretación
Funciona sin errores	A	Nivel suficiente consignado por el equipo; estructura de ejecución disponible.
Cumplimiento del objetivo funcional	A	Implementa reservas, simulaciones, estados y resultados previstos.
Diseño de la aplicación	B	Interfaz funcional con oportunidades de mejora en presentación y controles.

TABLA VI. RÚBRICA DE CONCURRENCIA: CALIFICACIONES REGISTRADAS

Criterio	Nivel	Fundamento técnico observable
Sincronización	A	Cola segura y protección atómica de consulta y cambio de estado.
Ausencia de deadlocks	A	Un Lock de asiento por operación y liberación mediante with.
Condiciones de carrera	A	No aparecen reservas duplicadas en las verificaciones realizadas.
Exclusión mutua	A	Bloqueo individual del asiento y protección de contadores.

Las letras A, B y C expresan niveles de logro, no puntajes numéricos únicos. Las rúbricas establecen valores máximos "hasta" 40/20/0 o 20/10/0 en funcionalidad, y 25/15/0 por criterio de concurrencia; la hoja completada por el equipo registra únicamente las categorías. Por lo tanto, no se asigna arbitrariamente una nota final sobre 100. La evaluación de la interfaz corresponde al nivel B que efectivamente figura en el Excel [3]–[5].

E. Limitaciones y objetivos de mejora

La revisión identificó aspectos que conviene mencionar para no sobrestimar el alcance del prototipo. Los identificadores de solicitudes se generan mediante números aleatorios, por lo que no existe una garantía estricta de unicidad. El número de workers se establece inicialmente en cuatro dentro de main.py, pero no aparece como parámetro ajustable desde la ventana. Asimismo, el cierre llama a la detención de productores y consumidores sin un paso explícito de espera por todos los productores activos antes de detener el sistema; este caso requiere pruebas específicas bajo cargas prolongadas [2].

Otra mejora recomendable consiste en centralizar toda interacción con Tkinter exclusivamente en el hilo principal: la aplicación ya utiliza una cola para resultados, pero determinados avisos de finalización se programan con after() desde los hilos secundarios. También debe aislarse la prueba que reinicia una función si se habilitan simulaciones simultáneas sobre el mismo catálogo. Ninguno de estos puntos invalida los resultados observados en las pruebas acotadas, pero delimita la solidez que puede atribuirse a una versión de producción [2].

Como pasos siguientes se propone adoptar identificadores monotónicos o UUID, permitir la reconfiguración segura de workers desde la interfaz, consolidar el protocolo de cierre, registrar métricas por corrida y repetir pruebas con distintas semillas y volúmenes de solicitudes. Una comparación con desarrollo manual solo sería válida si se dispusiera de una implementación equivalente y de tiempos y condiciones medidos bajo un protocolo común.

V. CONCLUSIONES

Cinema Reservation System constituye una implementación didáctica del patrón productor–consumidor aplicada a un problema realista de inventario compartido. La arquitectura separa generación y procesamiento de solicitudes, utiliza colas sincronizadas y protege la modificación de cada asiento mediante un bloqueo individual. En las verificaciones realizadas, las 600 peticiones de cada escenario fueron procesadas sin detectar ventas duplicadas, y tres ejecuciones de la prueba de conflicto arrojaron exactamente una aceptación y 19 rechazos.

Las mediciones asentadas por el equipo reflejan uso medio acotado de CPU y un pico de memoria de 35 MB para ambas acciones consideradas. Las rúbricas registran desempeño de nivel A en funcionalidad general y en los cuatro criterios de concurrencia, con nivel B para el diseño de la interfaz. Los datos disponibles respaldan la viabilidad de la solución en los escenarios estudiados, pero no permiten cuantificar una ventaja frente al desarrollo manual ni asegurar corrección absoluta bajo cargas o cierres no evaluados.

La principal enseñanza del trabajo es que un prompt técnico detallado puede orientar la generación asistida de una aplicación concurrente modular; sin embargo, la calidad de la solución se establece mediante inspección del código, evidencia experimental y análisis crítico de sus límites, no únicamente por la producción automática de una interfaz funcional.

VI. REFERENCIAS Y BIBLIOGRAFÍA

A. Referencias bibliográficas

[1] Equipo de desarrollo, “Prompt técnico cinema”, especificación interna de requisitos, archivo de texto suministrado para el trabajo práctico, 2026.

[2] Equipo de desarrollo, “Cinema Reservation System”, código fuente: main.py, models.py, cinema.py, reservation_system.py, simulation.py, ui.py y medir_metricas.py, archivo ZIP suministrado, 2026.

[3] Equipo de desarrollo, “Plantilla métrica 1”, hojas Datos del proyecto, Funcionalidad, CPU_Memoria, Concurrencia y Datos Tokens, planilla XLSX suministrada, 2026.

[4] Cátedra, “Rúbrica funcionalidad programa concurrente PC (Java/Python/Kotlin)”, documento de evaluación proporcionado para la actividad.

[5] Cátedra, “Rúbrica de concurrencia”, documento de evaluación proporcionado para la actividad.

[6] Python Software Foundation, “threading — Thread-based parallelism”, Python Documentation. [En línea]. Disponible: https://docs.python.org/3/library/threading.html. Consultado: 10-oct-2026.

[7] Python Software Foundation, “queue — A synchronized queue class”, Python Documentation. [En línea]. Disponible: https://docs.python.org/3/library/queue.html. Consultado: 10-oct-2026.

[8] G. Rodola, “psutil documentation”. [En línea]. Disponible: https://psutil.readthedocs.io/. Consultado: 10-oct-2026.

B. Bibliografía complementaria

No se incorporó bibliografía adicional a las fuentes citadas en el cuerpo del informe.